

Assistente de Pesquisa de Engenharia de Dados

Diagnóstico, correção e validação de uma rotina de coleta de resultados de busca do G1 para o termo LGPD

Fernanda Fregulha | Vaga: Assistente de Pesquisa de Engenharia de Dados

Repositório: <https://github.com/fregulha/netlab-g1-webscraping.git>

Projeto: NetLab G1 — Diagnóstico e correção de coleta de resultados de busca

Resumo executivo

Este projeto investigou uma falha na rotina de coleta de resultados de busca do portal G1 para o termo “LGPD”. A rotina original deixou de produzir resultados confiáveis após mudanças na estrutura HTML, na lógica de paginação e no comportamento dinâmico do portal. O objetivo foi diagnosticar a causa raiz, corrigir a rotina em Python com BeautifulSoup e validar a solução com testes automatizados e métricas de qualidade.

A solução desenvolvida se baseia em parsing defensivo, normalização de URLs, deduplicação, tratamento de erros, exportação em CSV e JSON e revisão de qualidade dos dados. A implementação foi organizada para ser legível, reutilizável e rastreável, atendendo ao contexto de pesquisa e engenharia de dados exigido pela vaga.

1. Contexto do problema

Recentemente, a coleta de resultados da busca do G1 para “LGPD” passou a devolver poucos resultados, resultados incompletos ou registros vazios, mesmo sem interromper a execução do código. O problema não era apenas um seletor incompatível: a própria página passou a carregar parte dos resultados de forma dinâmica, o que afetou a premissa de que o HTML inicial continha o conjunto completo de itens.

2. Diagnóstico técnico

A evidência observada diretamente no navegador mostra que a página de busca do G1 não entrega a totalidade dos resultados em uma única carga HTML estática. Ao rolar a página, o portal dispara requisições e vai montando novos itens dinamicamente na interface. Esse comportamento explica a regressão observada na rotina original: a coleta baseada em HTML bruto da primeira resposta passou a retornar dados parciais, incompletos ou vazios.

Além disso, a rotina original apresentava outros problemas relevantes: seletores incompatíveis, ausência de validadores de resposta HTTP, campos vazios sem tratamento, sobrescrita de resultados, ausência de deduplicação e baixa rastreabilidade operacional.

3. Estratégias adotadas

- revisão de HTML e seletores com base em snapshots reais do portal
- normalização de URLs e remoção de parâmetros de rastreamento
- deduplicação por URL para evitar registros repetidos
- tratamento de falhas de conexão, HTTP inválido e campos ausentes
- exportação em CSV e JSON com rastreabilidade de página, snapshot e horário da coleta
- validação automatizada com testes para componentes críticos do scraper

4. Solução implementada

A rotina foi reorganizada em módulos com responsabilidades bem definidas, permitindo maior legibilidade e reutilização. O scraper passou a validar a resposta da página, tratar erros de rede, separar registros válidos e em quarentena, exportar resultados estruturados e registrar sinais de diagnóstico durante a execução.

- coleta, parsing e normalização do conteúdo extraído
- validação de URL, dados ausentes e resposta HTTP
- deduplicação e preservação da primeira ocorrência válida
- exportação em formatos estruturados com informações de origem
- métricas e avaliação de qualidade dos dados coletados

5. Seletores e paginação

- Card de notícia: `li.widget--info`
- Card de vídeo: `li.video-widget--info`
- Título: `.widget--info__title`
- Resumo: `.widget--info__description`
- Data exibida: `.widget--info__meta`
- URL: link associado ao card (`a[href]`)
- Mais resultados: `button.pagination__load-more`

A lógica de paginação foi tratada como parte do problema: a página do G1 não se comporta como uma página estática simples, mas como um ambiente de carregamento incremental. Essa observação foi incorporada à documentação e ao diagnóstico da solução.

6. O que foi feito

O projeto foi conduzido em três frentes principais: diagnóstico técnico, correção da rotina e validação dos dados. Primeiro, analisamos a causa raiz da regressão e identificamos que a página de busca do G1 deixou de ser tratada como HTML estático completo. Em seguida, reorganizamos a lógica de extração para lidar com respostas parciais, URLs inválidas, campos ausentes e registros repetidos. Por fim, validamos a qualidade dos dados com amostra de referência, métricas explícitas e testes automatizados.

- análise da falloff do processo de coleta e revisão dos seletores
- refatoração do scraper para parsing defensivo e modular
- normalização de URLs, deduplicação e separação de registros em quarentena
- tratamento de erros de rede, HTTP e dados incompletos sem interromper a execução
- exportação em CSV e JSON com metadados de origem, página e coleta
- documentação de instalação, execução e diagnóstico do projeto
- proposta de uso de LLM como apoio humano à análise de HTML e testes de regressão
- avaliação quantitativa da qualidade da coleta com indicadores de cobertura, unicidade e consistência

7. Métricas e evidências de qualidade

- 25 testes automatizados aprovados com pytest
- 30 URLs únicas após deduplicação
- 26 registros de notícias e 4 de vídeos

- título, URL, resumo e data exibida presentes em 30/30 registros
- rastreabilidade de página, snapshot, origem e horário de coleta preservada
- dados de publicação extraídos somente quando houver evidência real em metadados

8. Uso de LLM

A LLM foi proposta como apoio ao diagnóstico e à manutenção da rotina, especialmente em momentos em que a página muda de estrutura ou quando se deseja comparar snapshots e logs de execução. A ideia não é automatizar a coleta nem confiar cega na resposta do modelo, mas sim usar a IA para analisar trechos de HTML, sugerir seletores e identificar inconsistências antes de qualquer alteração no código.

As respostas do modelo precisam ser validadas com evidências coletadas, testes automatizados e comparação com referência manual. Esse controle é importante para garantir que a rotina continue extraindo apenas dados realmente presentes na página e que não haja alucinação de campos ou metainformações inexistentes.

9. Limitações e próximos passos

- a coleta completa do portal G1 pode depender de scroll e requisições dinâmicas
- a página pode sofrer mudanças estruturais e exigir revisão periódica dos seletores
- a data de publicação deve ser confirmada em metadados do artigo e não inferida sem evidência
- o próximo passo mais robusto é monitorar a API ou o fluxo de carregamento real do site com automação de navegador

10. Conclusão

A solução desenvolvida recuperou a robustez da rotina de coleta, preservou a qualidade dos dados e documentou com clareza o processo de diagnóstico e correção. O resultado atende aos requisitos do edital e demonstra preparado para atuar com rigor em tarefas de engenharia de dados, análise de dados e automação de coleta na web.

Links e referências

GitHub: <https://github.com/fregulha/netlab-g1-webscraping.git>

Portal de referência: <https://g1.globo.com/busca/?q=lgpd>

Validação: 25 testes automatizados aprovados com pytest